

K-Nearest Neighbors (KNN)

A Practical Guide to Instance-Based Machine Learning

GitHub-ready ML Notes

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. Instead of learning an explicit mathematical model during training, KNN stores training data and makes predictions by finding the most similar observations, or nearest neighbors, to a new data point.

1. What is KNN?

KNN predicts an unknown data point using the labels or values of the K closest training observations. The value of K determines how many neighbors participate in the prediction.

2. How KNN Works

```
New Data Point
↓
Calculate distance to training points
↓
Sort points by distance
↓
Select K nearest neighbors
↓
Classification → Majority vote
Regression    → Average neighbor values
↓
Final Prediction
```

3. Choosing K

K is the number of neighbors considered during prediction. A very small K can make the model sensitive to noise, while a very large K can oversimplify the decision boundary.

K	Typical Effect
K = 1	Very flexible; sensitive to noise
Small K	Low bias, higher variance
Large K	Higher bias, lower variance
Very large K	May oversimplify the data

4. Distance Metrics

KNN depends on a distance or similarity measure. Euclidean distance is commonly used.

Euclidean Distance

$$d = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]}$$

Manhattan Distance

$$d = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

5. Why Feature Scaling is Important

KNN is distance-based, so features with large numerical ranges can dominate the distance calculation. Scaling makes features more comparable.

```
z = (x - μ) / σ

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

6. KNN Classification

For classification, KNN generally uses majority voting. If K=5 and three of the five nearest neighbors belong to Class 1, the new observation is classified as Class 1.

```
Neighbors = [1, 1, 1, 0, 0]
Prediction = 1
```

7. KNN Regression

For regression, KNN commonly predicts the average target value of the nearest neighbors.

```
Neighbor values = [20, 24, 22, 26, 23]
Prediction = 23
```

8. KNN in Python — Classification

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv('data.csv')

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

9. Important KNN Parameters

n_neighbors: Number of neighbors.

weights: 'uniform' gives equal weight; 'distance' gives closer neighbors more influence.

metric: Distance metric such as Euclidean or Manhattan.

p: Parameter for Minkowski distance; p=2 corresponds to Euclidean distance.

```
KNeighborsClassifier(  
    n_neighbors=5,  
    weights='distance',  
    metric='minkowski',  
    p=2  
)
```

10. Hyperparameter Tuning

The best K is data-dependent. Cross-validation can be used to compare multiple values of K and other parameters.

```
from sklearn.model_selection import GridSearchCV  
  
param_grid = {  
    'n_neighbors': range(1, 21),  
    'weights': ['uniform', 'distance'],  
    'metric': ['euclidean', 'manhattan']  
}  
  
grid = GridSearchCV(  
    KNeighborsClassifier(), param_grid, cv=5, scoring='accuracy'  
)  
grid.fit(X_train, y_train)  
  
print(grid.best_params_)  
print(grid.best_score_)
```

11. Overfitting and Underfitting

Small K: complex decision boundary, potentially higher variance and overfitting.

Large K: smoother decision boundary, potentially higher bias and underfitting.

```
Small K → Complex boundary → Higher variance  
Large K → Smooth boundary → Higher bias
```

12. Evaluation Metrics

For classification, use accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC where appropriate. For regression, common metrics include MAE, MSE, RMSE, and R².

13. Advantages

- Simple and intuitive
- Easy to implement
- Works for classification and regression
- Can model nonlinear decision boundaries
- Naturally supports multiclass classification

14. Limitations

- Prediction can be slow on large datasets
- Sensitive to feature scaling
- Sensitive to irrelevant features
- Can struggle in high-dimensional spaces
- Choice of K strongly affects performance
- Requires storing the training dataset

15. Curse of Dimensionality

As the number of features increases, distance-based similarity can become less informative. This is known as the curse of dimensionality. Feature selection or dimensionality reduction can sometimes improve KNN performance.

16. KNN vs Logistic Regression

Feature	KNN	Logistic Regression
Type	Instance-based	Coefficient-based
Main task	Classification & regression	Mainly classification
Training	Minimal model fitting	Learns coefficients
Prediction	Distance-based	Probability-based
Scaling	Usually important	Often useful
Nonlinear patterns	Can handle naturally	Often needs feature engineering

17. Practical Machine Learning Workflow

```
Define Problem
↓
Collect Data
↓
EDA + Cleaning
↓
Encoding + Feature Engineering
↓
Train-Test Split
↓
Feature Scaling
↓
Train KNN
↓
Tune K + Distance Parameters
↓
Evaluate Model
↓
Interpret + Document Results
```

18. Common Mistakes

- Skipping feature scaling when feature ranges differ substantially
- Choosing K without validation
- Using many irrelevant features
- Evaluating only on training data
- Ignoring class imbalance
- Fitting the scaler on the complete dataset before splitting
- Ignoring the curse of dimensionality

19. Key Takeaways

```

New Point
  ↓
Calculate Distances
  ↓
Find K Neighbors
  ↓
Vote / Average
  ↓
Prediction

```

KNN is a simple and intuitive algorithm based on the idea that similar observations tend to have similar outcomes. The most important concepts are K selection, distance metrics, scaling, voting, hyperparameter tuning, overfitting, and the curse of dimensionality.

20. Suggested GitHub Repository Structure

```

k-nearest-neighbors/
├── data/
│   └── dataset.csv
├── notebooks/
│   └── knn_analysis.ipynb
├── images/
│   ├── confusion_matrix.png
│   └── model_performance.png
├── README.md
├── requirements.txt
└── K_Nearest_Neighbors_Guide.pdf

```

For a stronger portfolio project, include EDA, preprocessing, several K values, cross-validation, distance-metric comparison, evaluation metrics, visualizations, and a conclusion explaining why the final model was selected.